Linear Equation System Manager

Comprehensive Final Project Documentation

Project Team:

Sama Yasser Gemeay	**12265**
Hager Sherif Ibrahim	**12266**
Rabea Shaban Ibrahim	**11219**

Table of Contents

1. Introduction
2. Problem Definition
3. Project Objectives
4. Detailed Project Description
5. Project Beneficiaries
6. System Analysis
7. Task Breakdown Structure
8. Time Plan & Roles
9. System Architecture and Design
10. Algorithms & Mathematical Techniques
11. System Flowchart
12. UML Class Diagram
13. Testing Scenarios and Results
14. End-User Guide
15. Limitations
16. Future Improvements
17. Conclusion
18. References

1. Introduction

This project presents a complete C++ console-based system designed to manage and solve systems of linear equations. The system applies Object-Oriented Programming principles and advanced matrix mathematics to provide a structured and maintainable solution.

2. Problem Definition

Solving systems of linear equations manually can be time-consuming and error-prone, especially when dealing with multiple variables. The objective of this project is to develop a computational system capable of parsing, analyzing, and solving such systems automatically.

3. Project Objectives

• Automatically detect and manage variables.

• Parse user-input equations dynamically.

• Construct coefficient matrices.

• Compute determinant values.

• Solve systems using Cramer's Rule.

• Apply substitution and equation operations.

• Maintain modular and scalable architecture.

4. Detailed Project Description

The system allows the user to enter multiple equations in standard mathematical form. The Equation class stores coefficients and constants, while the EquationManager class handles parsing, matrix construction, determinant calculation, and solving operations.

5. Project Beneficiaries

• Mathematics students.

• Engineering students.

• Researchers dealing with linear models.

• Learners studying OOP implementation in C++.

6. System Analysis

Functional Requirements:

- Accept equation input.
- Parse coefficients and constants.
- Perform determinant calculations.
- Solve systems correctly.

Non-Functional Requirements:

- High precision.
- Efficient memory usage.
- Clean architecture.

7. Task Breakdown Structure

- Requirement Analysis
- System Design
- Algorithm Design
- Implementation
- Testing
- Documentation
- Presentation Preparation

8. System Architecture and Design

The system follows modular OOP architecture consisting of:

• Equation Class: Handles equation-level operations.

• EquationManager Class: Handles system-level logic and matrix operations.

9. Algorithms & Mathematical Techniques

• Matrix representation of linear systems.

• Determinant calculation using recursive expansion.

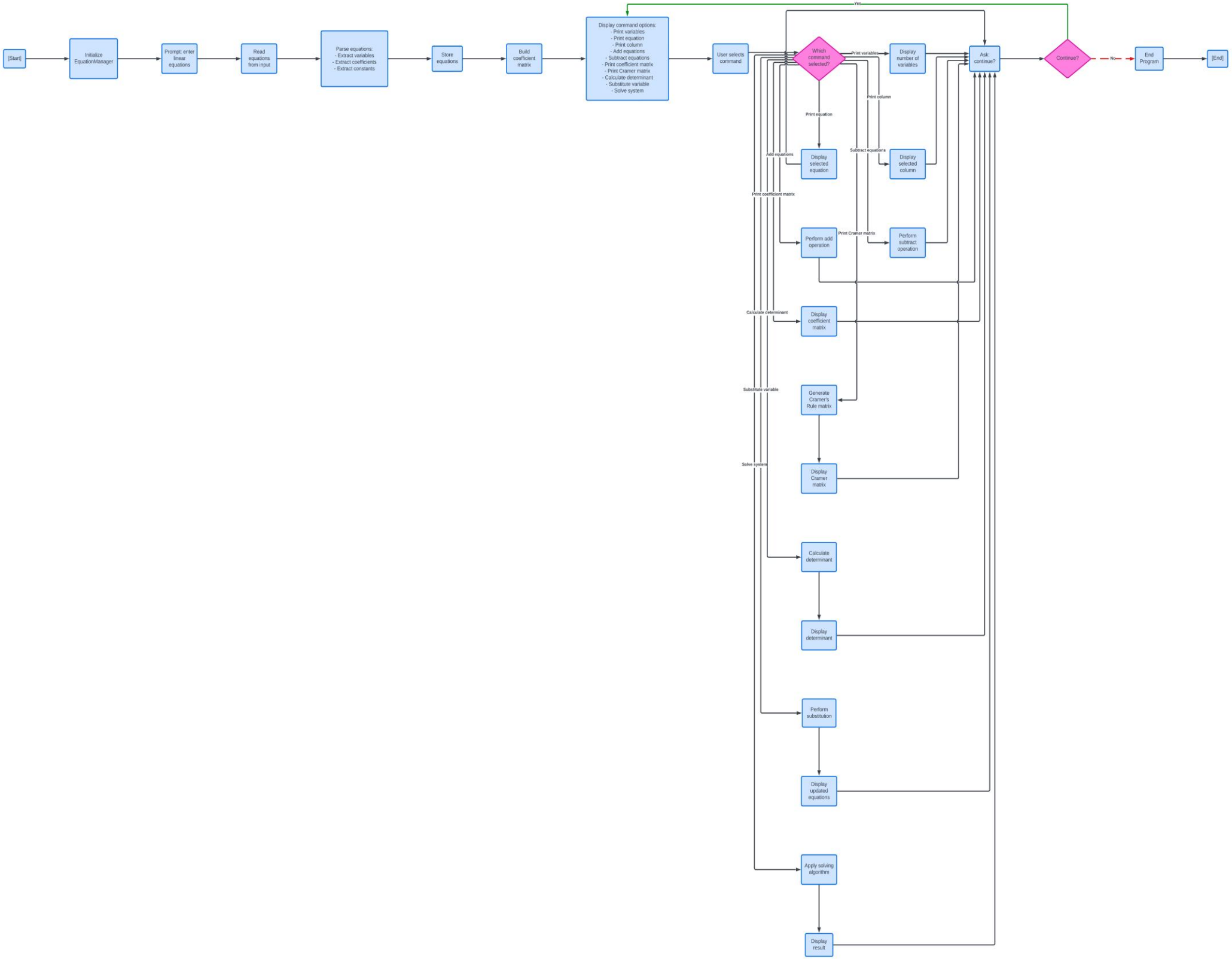
• Cramer's Rule implementation.

• Floating-point precision handling using fabs().

10. System Flowchart

The following flowchart illustrates the overall workflow of the **Linear Equation Solver** system.
It shows the sequence of operations starting from user input of the number of equations and coefficients, followed by creating equation objects, storing them in the equation manager, solving the system of equations, and finally displaying the results.

This diagram helps visualize how the program processes data and how the different steps are connected to achieve the final solution.


[Start]
Initialize EquationManager
Prompt: enter linear equations
Read equations from input
Parse equations:
- Extract variables
- Extract coefficients
- Extract constants
Store equations
Build coefficient matrix
Display command options:
- Print variables
- Print equation
- Print column
- Add equations
- Subtract equations
- Print coefficient matrix
- Print Cramer matrix
- Calculate determinant
- Substitute variable
- Solve system
User selects command
Which command selected?
Print variables
Display number of variables
Print column
Print equation
Display selected equation
Add equations
Subtract equations
Display selected column
Print coefficient matrix
Perform add operation
Print Cramer matrix
Perform subtract operation
Calculate determinant
Display coefficient matrix
Substitute variable
Generate Cramer's Rule matrix
Solve system
Display Cramer matrix
Calculate determinant
Display determinant
Perform substitution
Display updated equations
Apply solving algorithm
Display result
Ask: continue?
Continue?
Yes
No
End Program
[End]

11. UML Class Diagram

The UML Class Diagram represents the structural design of the **Linear Equation Solver** system.
It illustrates the main classes used in the program, their attributes, methods, and the relationships between them.

The system is mainly composed of two core classes:

- **Equation** – represents a single linear equation.
- **EquationManager** – manages multiple equations and performs the solving process.

The diagram helps in understanding how the program is organized using **Object-Oriented Programming (OOP)** principles such as encapsulation and modular design.

UML Class Diagram

Linear Equation System Manager - UML Class Diagram

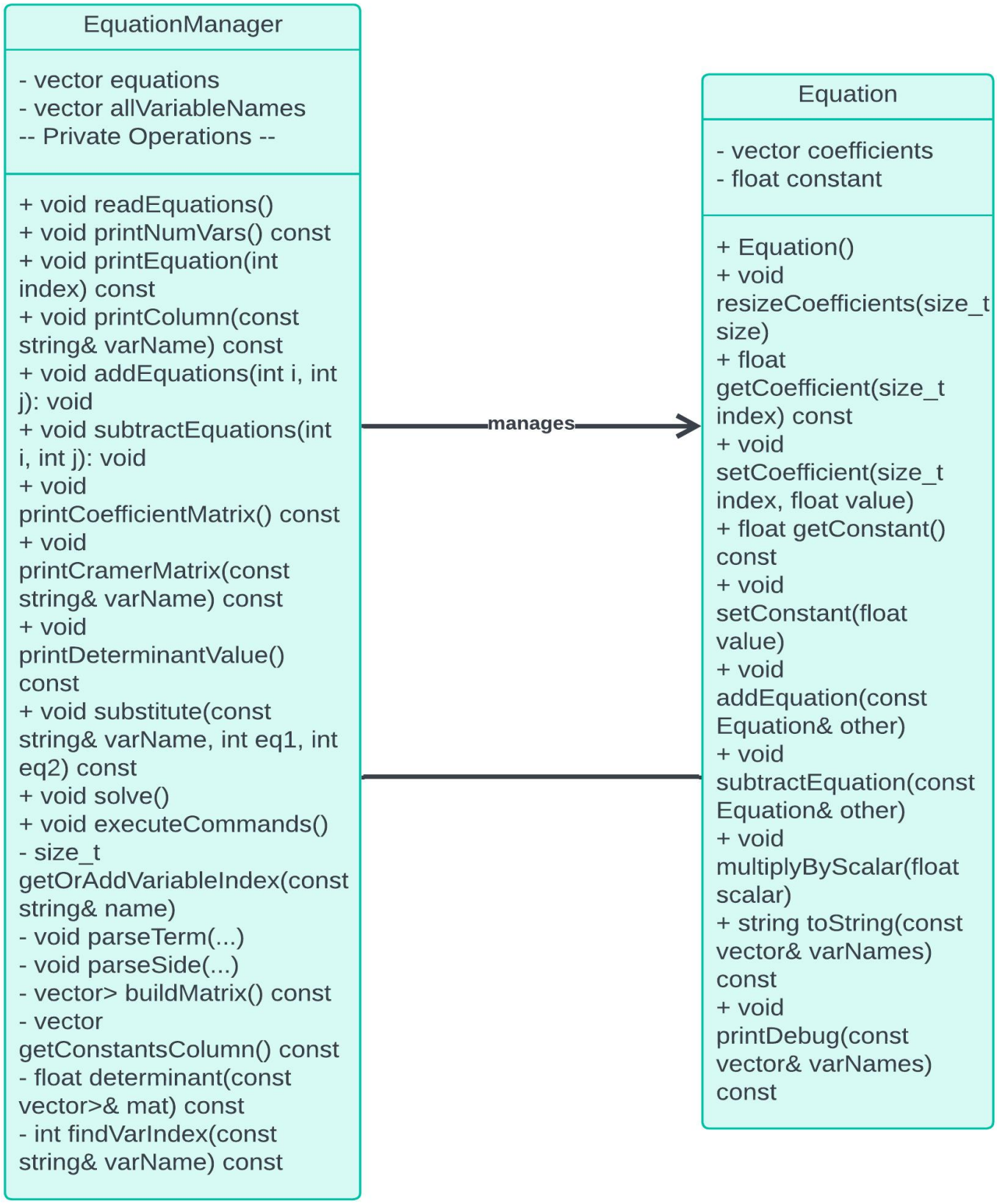


Diagram Explanation

The **Equation** class is responsible for representing a single linear equation.
It stores the coefficients of the variables and the constant value of the equation.
This class also provides functions for inputting and displaying the equation.

The **EquationManager** class manages a collection of equations.
It stores multiple Equation objects and provides methods to add equations, solve the system of linear equations, and display the results.

The relationship between these two classes is a **composition relationship**, where the EquationManager contains and manages multiple Equation objects.

12.Testing Scenarios and Results

Test Case 1: Unique Solution – Passed.

Test Case 2: Infinite Solutions – Passed.

Test Case 3: No Solution – Passed.

13.End-User Guide

1. Run the application.

2. Enter equations.

3. Choose operation (solve, determinant, print matrix, substitute).

4. View results in formatted form.

14.Limitations

• Floating-point rounding limitations.

• Recursive determinant inefficiency for large matrices.

15.Future Improvements

- Implement Gaussian Elimination.
- Optimize determinant performance.
- Develop GUI version.
- Add step-by-step solving visualization.

16.Conclusion

The project successfully integrates mathematical theory with practical software development principles. It demonstrates strong OOP design, matrix manipulation capabilities, and accurate system solving.

17.References

1. Elementary Linear Algebra – Howard Anton.
2. C++ ISO Documentation.
3. Numerical Methods for Engineers – Steven C. Chapra.